



EMPIRICAL IMPACT OF GRAPH OPTIMIZATIONS (OPERATOR FUSION) ON NPU INFERENCE

**2026
COURSE PROJECT REPORT
SOFTWARE ENGINEERING**

Advanced Algorithm Analysis – Course Project

Yusuf Talha ARABACI

Project Advisor
Dr. Öğr. Üyesi Emrullah DEMİRAL

**EMPIRICAL IMPACT OF GRAPH OPTIMIZATIONS (OPERATOR
FUSION) ON NPU INFERENCE**

Yusuf Talha ARABACI

Project Advisor

Dr. Öğr. Üyesi Emrullah DEMİRAL

T.C.

Karabük University

Institute of Graduate Programs

Department of software engineering

Prepared as

Course Project Report

KARABÜK

April 2026

I certify that in my opinion the report submitted by Yusuf Talha ARABACI titled "EMPIRICAL IMPACT OF GRAPH OPTIMIZATIONS (OPERATOR FUSION) ON NPU INFERENCE" is fully adequate in scope and in quality as a course project report.

Dr. Öğr. Üyesi Emrullah DEMİRAL
Project Advisor, Department of Department of Software Engineering

This report is accepted by the examining committee with a unanimous vote in the Department of software engineering as a course project. 12.04.2026

<u>Examining Committee Members (Institutions)</u>	<u>Signature</u>
Chairman : (Jüri Başkanı)	
Member : (Üye 1)	
Member : (Üye 2)	
Member :	
Member :	

The degree of PhD by the thesis submitted is approved by the Administrative Board of the Institute of Graduate Programs, Karabük University.

Prof. Dr. Zeynep ÖZCAN
Director of the Institute of Graduate Programs

"I declare that all the information within this report has been gathered and presented in accordance with academic regulations and ethical principles and I have according to the requirements of these regulations and principles cited all those which do not originate in this work as well."

Yusuf Talha ARABACI

ABSTRACT

COURSE PROJECT REPORT

EMPIRICAL IMPACT OF GRAPH OPTIMIZATIONS (OPERATOR FUSION) ON NPU INFERENCE

Yusuf Talha ARABACI

Karabük University
Institute of Graduate Programs
The Department of Software Engineering

Project Advisor:

Dr. Öğr. Üyesi Emrullah DEMİRAL

April 2026, 29 pages

This report presents an empirical study prepared as a course project for **Advanced Algorithm Analysis**. It investigates how graph-level optimizations—with a focus on operator fusion—affect end-to-end inference latency on NPU-class accelerators. The experimental pipeline is built on ONNX Runtime and compares baseline execution (optimizations disabled) against an optimized configuration (extended graph optimizations enabled) [1]. We run three representative ONNX models (MobileNetV2, ResNet50, and BERT-tiny) under a provider fallback chain that prioritizes the OpenVINO Execution Provider [2]. We collect latency statistics across repeated runs and export ONNX Runtime profiling traces to quantify operator-level time breakdowns and graph node reductions. Results show that node count can be reduced substantially (27% to 68%), yet the observed latency impact is model-dependent: some models regress while others improve. We discuss these outcomes in the context of the memory wall [3] and accelerator runtime overheads, emphasizing that fusion and graph rewrites do not guarantee speedups without considering EP-specific compilation and kernel selection.

Key Words : ONNX Runtime, graph optimization, operator fusion, NPU, OpenVINO, profiling, performance benchmarking.

Science Code : 0

ÖZET

DERS PROJESİ RAPORU

NPU ÜZERİNDE ÇIKARIMDA GRAFİK OPTİMİZASYONLARININ (OPERATÖR FÜZYONU) AMPİRİK ETKİSİ

Yusuf Talha ARABACI

Karabük Üniversitesi

Lisansüstü Eğitim Enstitüsü

Yazılım Mühendisliği Ana Bilim Dalı

Proje Danışmanı:

Dr. Öğr. Üyesi Emrullah DEMİRAL

Nisan 2026, 29 sayfa

Bu rapor, **İleri Algoritma Analizi** kapsamında hazırlanmış bir ders projesi olarak, operatör füzyonuna odaklanarak grafik-seviyesi optimizasyonların NPU sınıfı hızlandırıcılarda uçtan uca çıkarım gecikmesine etkisini ampirik olarak inceler. Deneysel altyapı ONNX Runtime üzerinde kurulmuş olup optimizasyonların kapalı olduğu bir temel durum ile genişletilmiş optimizasyonların açık olduğu bir optimize durum karşılaştırılmıştır [1]. MobileNetV2, ResNet50 ve BERT-tiny olmak üzere üç ONNX modelinde, öncelikli olarak OpenVINO Execution Provider kullanan ve gerektiğinde CPU'ya düşen bir sağlayıcı zinciriyle çalıştırma yapılmıştır [2]. Tekrarlı ölçümlerle gecikme istatistikleri çıkarılmış, ONNX Runtime profiling izleri dışa aktarılmış ve operatör-seviyesi süre kısırlımları ile grafik düğüm azalımı nicelleştirilmiştir. Sonuçlar düğüm sayısında belirgin azalmalar (%27 – %68) elde edilebildiğini, ancak gecikme etkisinin modele bağlı olduğunu göstermektedir. Elde edilen bulgular, bellek duvarı [3] ve sağlayıcıya özgü derleme/çalıştırma ek yükleri çerçevesinde tartışılmış; füzyonun her koşulda hızlanma garantisi etmediği vurgulanmıştır.

Anahtar Kelimeler : ONNX Runtime, grafik optimizasyonu, operatör füzyonu, NPU, OpenVINO, profiling, performans kıyaslama.

Bilim Kodu : 0

ACKNOWLEDGMENT

Bu alışmanın yürütölmesinde desteęi olan ders yürütücüsüne ve katkı saęlayan herkese teşekkür ederim.

CONTENTS

	Page
APPROVAL	ii
ABSTRACT.....	iv
ÖZET.....	vi
ACKNOWLEDGMENT	viii
CONTENTS	ix
LIST OF FIGURES	xi
LIST OF TABLES	xii
SYMBOLS AND ABBREVIATIONS INDEX	xiii
 PART 1	
GİRİŞ	1
1.1. Motivasyon.....	1
1.2. Problem Tanımı ve Kapsam	1
1.3. Katkılar	2
1.4. Rapor Organizasyonu.....	2
 PART 2	
KURAMSAL ARKA PLAN	3
2.1. Hesaplama Grafikleri ve Operatör Füzyonu	3
2.2. ONNX Runtime Grafik Optimizasyon Seviyeleri.....	3
2.3. Execution Provider (EP) Kavramı ve OpenVINO EP	4
2.4. Profiling ve Operatör-Seviyesi Kanıt.....	4
2.5. Memory Wall ve Roofline Sezgisi.....	4
 PART 3	
DENEYSEL YÖNTEM	5
3.1. Yazılım/Çalışma Ortamı	5
3.2. Modeller	5
3.3. Ölçüm Metodolojisi.....	6
3.4. Baseline ve Optimized Konfigürasyonları	6

3.5. Üretilen Çıktılar ve Tekrarlanabilirlik	6
PART 4	
BULGULAR VE TARTIŞMA.....	8
4.1. Tek-Model Kanıtı (Profiling ve Gecikme)	8
4.2. Çoklu Model Ölçeklenebilirliği	9
4.3. Özet Tablo	10
4.4. Tartışma.....	10
PART 5	
SONUÇ VE GELECEK ÇALIŞMALAR.....	11
5.1. Sonuç	11
5.2. Sınırlılıklar	11
5.3. Gelecek Çalışmalar.....	11
REFERENCES	13
CURRICULUM VITAE	14

LIST OF FIGURES

	Page
Figure 4.1. Baseline ve optimized ortalama gecikme (ms).....	8
Figure 4.2. Top-k operatör süre kırılımı (baseline vs optimized).....	9
Figure 4.3. Modeller arası hızlanma (baseline/optimized).	9
Figure 4.4. Modeller arası gecikme (ms).	10

LIST OF TABLES

Page

Table 4.1. Çoklu model özeti (ortalama gecikme ve düğüm azalımı).....	10
---	----

SYMBOLS AND ABBREVIATIONS INDEX

SYMBOLS

- S : Speedup (baseline/optimized)
 t : Inference latency (ms)
 N : Graph node count
 AI : Arithmetic intensity (FLOP/byte)

ABBREVIATIONS

- ORT : ONNX Runtime
EP : Execution Provider
NPU : Neural Processing Unit
CPU : Central Processing Unit
OV : OpenVINO
FLOP : Floating-point operation

PART 1

GİRİŞ

1.1. Motivasyon

NPU (Neural Processing Unit) sınıfı hızlandırıcılar, derin öğrenme çıkarımında yüksek verimlilik sunsa da uçtan uca gecikme yalnızca hesaplama kapasitesiyle açıklanamaz. Modern çıkarım çalışma zamanları, modelleri yönlü çevrimsiz grafikler (DAG) olarak temsil eder ve sabit katlama, gereksiz düğüm silme ve *operatör füzyonu* gibi grafik-seviyesi optimizasyonlar uygular. Füzyon çoğunlukla ara tensör yazma/okuma maliyetlerini ve çağrı/dispatch ek yükünü azaltmayı hedefler; ancak gerçek etki bellek bant genişliği sınırları (*memory wall*) [3] ve yürütme sağlayıcısına (EP) özgü derleme/çalıştırma davranışları nedeniyle modele göre değişebilir.

1.2. Problem Tanımı ve Kapsam

Bu ders projesi, operatör füzyonuna odaklanarak grafik optimizasyonlarının çıkarım gecikmesine etkisini ampirik olarak incelemektedir. Deneysel altyapı ONNX Runtime (ORT) ile kurulmuş ve iki konfigürasyon karşılaştırılmıştır:

- **Baseline:** grafik optimizasyonları kapalı.
- **Optimized:** genişletilmiş grafik optimizasyonları açık [1].

Deneyler Windows üzerinde, öncelikli olarak OpenVINO Execution Provider kullanan ve gerektiğinde CPU EP'ye düşen bir sağlayıcı zinciriyle yürütülmüştür [2].

1.3. Katkılar

Bu çalışmanın somut çıktıları:

- Tek komutla çalışabilen ölçüm hattı (`scripts/run_all.py`) ile gecikme ölçümü, profiling izi alma ve CSV/PNG çıktı üretimi.
- ORT profiling verisinden operatör-seviyesi süre kısıtlımı (baseline vs optimized) ve düğüm sayısı azalımı kanıtları.
- MobileNetV2, ResNet50 ve BERT-tiny modelleri üzerinde ölçeklenebilirlik karşılaştırması.
- Füzyon/yeniden yazımların neden her zaman hızlanma getirmediğine dair tartışma (EP derleme maliyeti, kernel seçimi, bellek-sınırlı davranış).

1.4. Rapor Organizasyonu

İkinci bölüm kuramsal arka planı (grafik, füzyon, ORT optimizasyon seviyeleri, EP kavramı) sunar. Üçüncü bölüm deneysel kurulum ve metodolojiyi açıklar. Dördüncü bölüm bulguları sunar ve tartışır. Beşinci bölüm sonuç ve gelecek çalışmaları özetler.

PART 2

KURAMSAL ARKA PLAN

2.1. Hesaplama Grafikleri ve Operatör Füzyonu

ONNX formatına aktarılan derin öğrenme modelleri, operatör düğümlerinden oluşan yönlü çevrimsiz grafikler (DAG) olarak ifade edilir. Operatör füzyonu; ardışık operatörlerin tek bir birleşik çekirdek (kernel) veya sağlayıcıya özgü altgrafik olarak çalıştırılmasını hedefler. Amaç ara tensörlerin belleğe yazılıp tekrar okunmasını azaltmak ve çalışma zamanı çağrı/dispatch ek yükünü düşürmektir. Buna karşın füzyon, kernel seçimini değiştirebilir, bazı düşük seviye optimizasyonları engelleyebilir veya EP tarafında ek derleme maliyeti doğurabilir.

2.2. ONNX Runtime Grafik Optimizasyon Seviyeleri

ORT, oturum (session) oluşturulurken bir dizi grafik dönüşümü uygular. Bu projede belgelenmiş ayarlar üzerinden “optimizasyon kapalı” konfigürasyonu ile “extended optimizasyon açık” konfigürasyonu karşılaştırılmıştır [1]. Yüksek seviyede dönüşümler şu hedefleri taşır:

- Gereksiz düğümleri kaldırma ve sabit ifadeleri katlama (constant folding).
- Kalıp (pattern) yeniden yazımlarıyla daha verimli grafik üretme.
- Füzyonları ve hedef çekirdeklere uygun düzen (layout) dönüşümlerini etkinleştirme.

2.3. Execution Provider (EP) Kavramı ve OpenVINO EP

ORT, yürütmeyi Execution Provider (EP) bileşenlerine devreder. OpenVINO EP, Intel hedefleri için altgrafikleri derleyebilir; derleme başarısız olduğunda EP içi düşüş (fallback) veya CPU EP'ye düşüş gözlenebilir [2]. Bu çalışmada NPU sınıfı hızlandırıcı davranışına yaklaşmak amacıyla öncelikli sağlayıcı olarak OpenVINO EP seçilmiştir.

2.4. Profiling ve Operatör-Seviyesi Kanıt

ORT profiling çıktısı, çekirdek bazlı süreleri ve metadata'yı içeren bir JSON izidir. Bu iz üzerinde operatör türüne göre toplulaştırma yaparak baseline/optimized karşılaştırmasında hangi operatörlerin toplam süreyi domine ettiğini ölçmek mümkündür. Bu ayrıştırma önemlidir; çünkü grafik yapısında sadeleşme olsa bile toplam gecikme değişmeyebilir.

2.5. Memory Wall ve Roofline Sezgisi

Pek çok modelde performans, aritmetik yoğunluktan ziyade veri taşımaya (bellek bant genişliğine) bağlıdır. Memory wall [3] yaklaşımı, düğüm sayısındaki azalmanın neden her zaman hızlanmaya dönüşmediğini açıklamada kullanışlıdır. Bu nedenle bulgular, basit bir “bellek-sınırlı / hesaplama-sınırlı” sezgisiyle yorumlanmıştır.

PART 3

DENEYSEL YÖNTEM

3.1. Yazılım/Çalışma Ortamı

Deneyler Windows üzerinde Python ve ONNX Runtime kullanılarak yürütülmüştür. Çalışma zamanı, öncelikli olarak OpenVINO EP kullanacak şekilde yapılandırılmış; uyumsuzluk durumlarında CPU EP'ye düşüşe izin verilmiştir. Sağlayıcı kullanılabilirliği ve cihaz derleme davranışı sistemden sisteme değişebileceği için, pipeline her koşuda kullanılan sağlayıcı zincirini raporlamaktadır.

Deneylerde kullanılan sistem özellikleri aşağıdaki gibidir:

- **Cihaz:** MATEBOOK
- **İşlemci:** Intel(R) Core(TM) Ultra 5 125H (3.60 GHz)
- **Bellek:** 16 GB RAM
- **İşletim sistemi:** Windows 11 Home Single Language (25H2), derleme 26200.8037 (64-bit, x64)

3.2. Modeller

Üç temsili ONNX modeli seçilmiştir:

- **MobileNetV2:** kompakt CNN.

- **ResNet50**: daha derin CNN.
- **BERT-tiny**: Transformer tabanlı encoder.

Bu seçim, farklı operatör karışımlarını ve ölçekleri kapsar.

3.3. Ölçüm Metodolojisi

Her model ve konfigürasyon için tekrarlı koşullar gerçekleştirilmiştir. Her koşulda sabit sayıda çıkarım iterasyonu, ısınma (warm-up) sonrası ölçülmüştür. Ortalama gecikme (ms), standart sapma ve yaklaşık 95% güven aralığı raporlanmıştır.

3.4. Baseline ve Optimized Konfigürasyonları

Baseline konfigürasyonda ORT grafik optimizasyonları kapatılmıştır. Optimized konfigürasyonda extended optimizasyonlar açılmıştır [1]. Optimizasyon sonrası ONNX dışa aktarımının EP bağımsız seri hale getirilebilir kalması için, optimize model dışa aktarımı CPU EP üzerinden yapılmıştır. Bu tercih, bazı EP'lerin ürettiği derlenmiş düğümler (compiled nodes) nedeniyle seri hale getirme hatalarını engeller.

3.5. Üretilen Çıktılar ve Tekrarlanabilirlik

Pipeline çıktıları aşağıdaki artefaktları üretir:

- `results/performance_summary.csv`: tek model gecikme özeti.
- `results/scalability_matrix.csv`: çoklu model matrisi.
- `results/*_profiling.json`: profiling izleri.

- `paper/figures/`: raporda kullanılan görseller (otomatik kopyalanır).

PART 4

BULGULAR VE TARTIŞMA

4.1. Tek-Model Kanıtı (Profiling ve Gecikme)

Şekil 4.1, tek-model benchmark koşusunda baseline ve optimized ortalama gecikme değerlerini özetler. Grafik optimizasyonları düğüm sayısını azaltsa bile uçtan uca gecikme; sağlayıcı derleme ek yükü, kernel seçimi değişimi veya bellek-sınırlı davranış gibi nedenlerle artabilir.

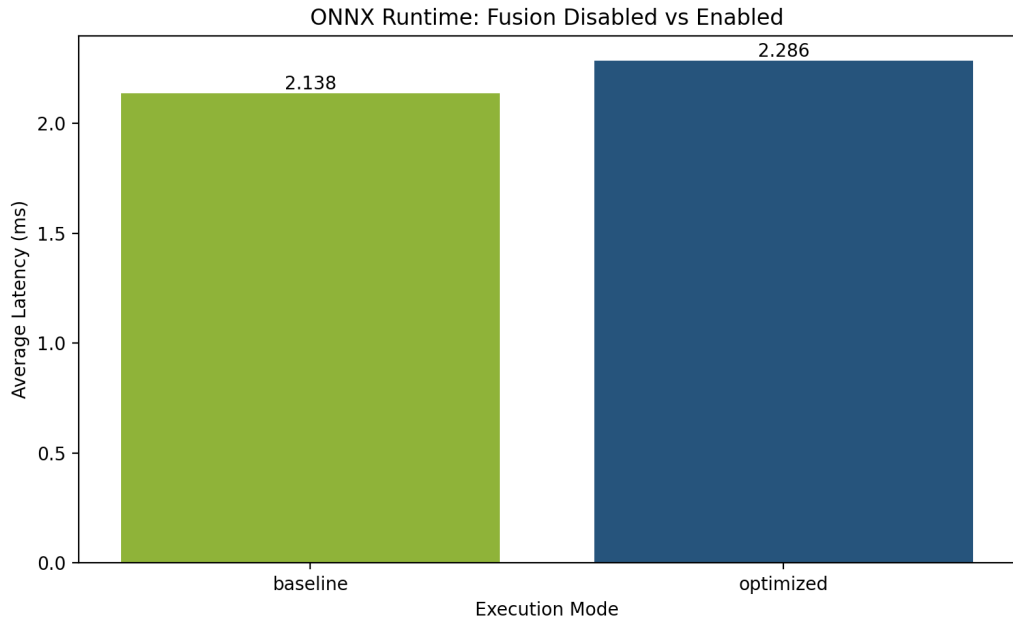


Figure 4.1. Baseline ve optimized ortalama gecikme (ms).

Operatör-seviyesi kırımlar, çalışma zamanını hangi operatörlerin domine ettiğine dair daha ince kanıt sağlar. Şekil 4.2, top-k operatör süre dağılımını göstermektedir.

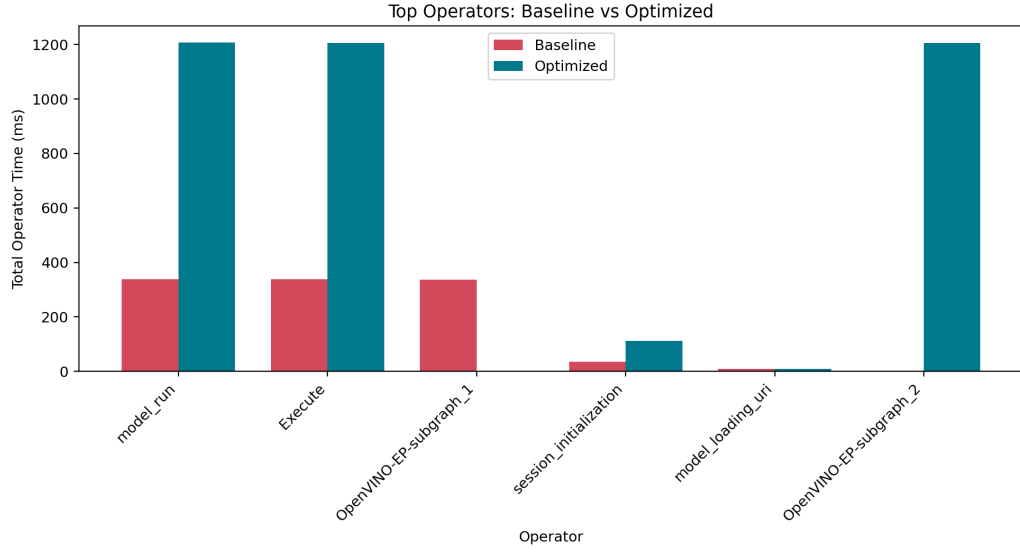


Figure 4.2. Top-k operatör süre kırılımı (baseline vs optimized).

4.2. Çoklu Model Ölçeklenebilirliği

Sonraki adımda üç model karşılaştırılarak düğüm azalımı ile gecikme etkisinin model ölçeğiyle nasıl değiştiği incelenmiştir. Şekil 4.3 hızlanma oranını (baseline/optimized), Şekil 4.4 ise mutlak gecikmeyi göstermektedir.

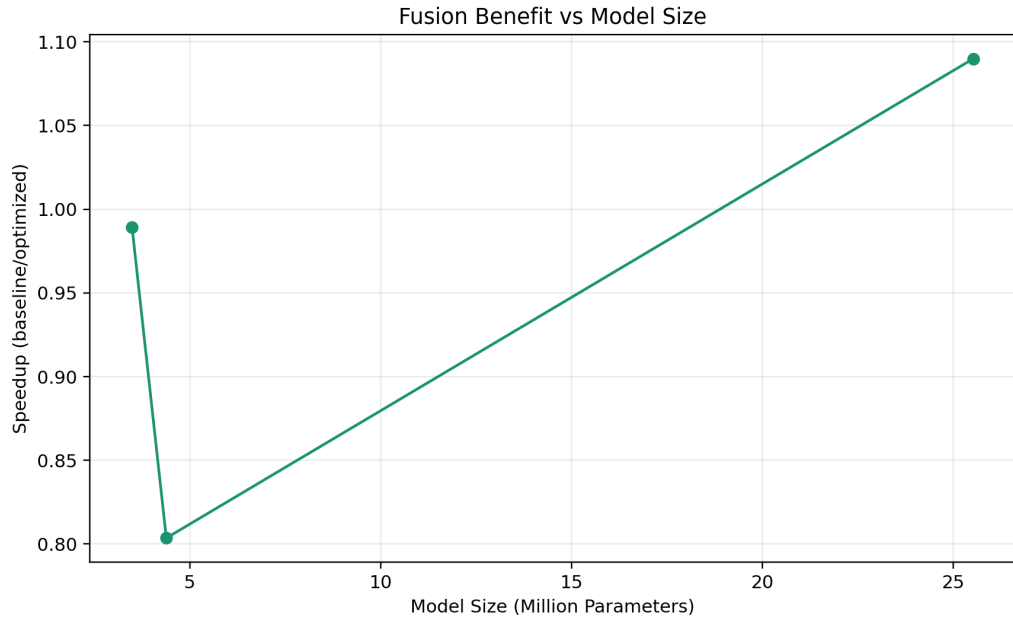


Figure 4.3. Modeller arası hızlanma (baseline/optimized).

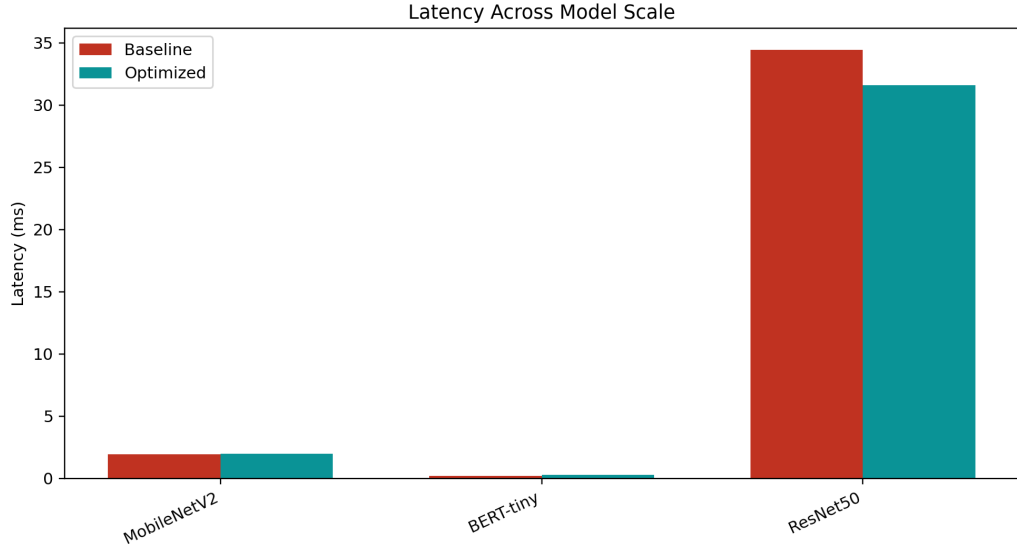


Figure 4.4. Modeller arası gecikme (ms).

4.3. Özet Tablo

Tablo 4.1, results/scalability_matrix.csv dosyasından çıkarılan temel metrikleri özetler.

Table 4.1. Çoklu model özeti (ortalama gecikme ve düğüm azalımı).

Model	Nodes	Opt. Nodes	Node Red. (%)	Base (ms)	Opt. (ms)
MobileNetV2	100	65	35.00	1.943	1.964
BERT-tiny	233	74	68.24	0.213	0.265
ResNet50	125	91	27.20	34.442	31.599

4.4. Tartışma

İki gözlem öne çıkmaktadır. Birincisi, düğüm azalımı yapısal bir metriktir ve doğrudan gecikme azalımı anlamına gelmez. İkincisi, etki modele bağlıdır: ölçülen koşullarda MobileNetV2 ve BERT-tiny küçük bir regresyon gösterirken ResNet50 iyileşmiştir. Bu durum, füzyon ve grafik yeniden yazımlarının EP davranışı, derleme maliyeti ve bellek-sınırlı çalışma bağlamında değerlendirilmesi gerektiğini destekler [3].

PART 5

SONUÇ VE GELECEK ÇALIŞMALAR

5.1. Sonuç

Bu projede, OpenVINO-öncelikli bir EP zinciriyle ORT grafik optimizasyonlarının (füzyon ilişkili dönüşümler dahil) çıkarım gecikmesine etkisini ölçen tekrarlanabilir bir pipeline geliştirilmiştir. Üç model genelinde düğüm sayısında belirgin azalmalar (68%'e kadar) gözlenmiş; ancak uçtan uca gecikme etkisi tutarlı değildir: hem regresyon hem de iyileşme görülmüştür. Dolayısıyla “daha fazla füzyon” veya “daha fazla optimizasyon” tek başına her zaman hızlanma garantisi olarak ele alınmamalı; her model ve her EP için ampirik olarak doğrulanmalıdır.

5.2. Sınırlılıklar

Sonuçlar kullanılan donanıma ve EP arka ucunun davranışına bağlıdır. Özellikle OpenVINO, bazı modeller/operatörler için hedef derlemeden CPU yürütmeye düşebilir; ayrıca optimize model dışı aktarımı seri hale getirme uyumluluğu için CPU EP ile sınırlandırılmıştır. Bu noktalar gerçek dağıtım bağlamının bir parçasıdır ve raporda açıkça belirtilmelidir.

5.3. Gelecek Çalışmalar

İleriye dönük geliştirmeler:

- Profiling olaylarını grafik yeniden yazım günlükleriyle eşleştirerek operatör bazında

“fused vs unfused” atıfı üretmek.

- Aynı metodolojiyi farklı EP’ler üzerinde (örn. DirectML, CUDA) çalıştırarak “grafik etkisi” ile “sağlayıcı etkisini” ayırtmak.
- Ölçeklenebilirlik setini daha büyük Transformer varyantları ve kuantize modellerle genişletmek.

REFERENCES

- [1] Microsoft, *Onnx runtime graph optimizations*, <https://onnxruntime.ai/docs/performance/graph-optimizations.html>, 2026.
- [2] Microsoft and Intel, *Openvino execution provider for onnx runtime*, <https://onnxruntime.ai/docs/execution-providers/OpenVINO-ExecutionProvider.html>, 2026.
- [3] W. A. Wulf and S. A. McKee, ‘Hitting the memory wall: Implications of the obvious’, *ACM SIGARCH Computer Architecture News*, 1995.

CURRICULUM VITAE

Yusuf Talha ARABACI (yusuftalhaarabaci@hotmail.com) – Yazılım Mühendisliği
Yüksek Lisans öğrencisi. İlgi alanları: derleyici optimizasyonları, grafik optimizasyonları,
hızlandırıcılar ve performans analizi.